

See what the code will do *differently*.

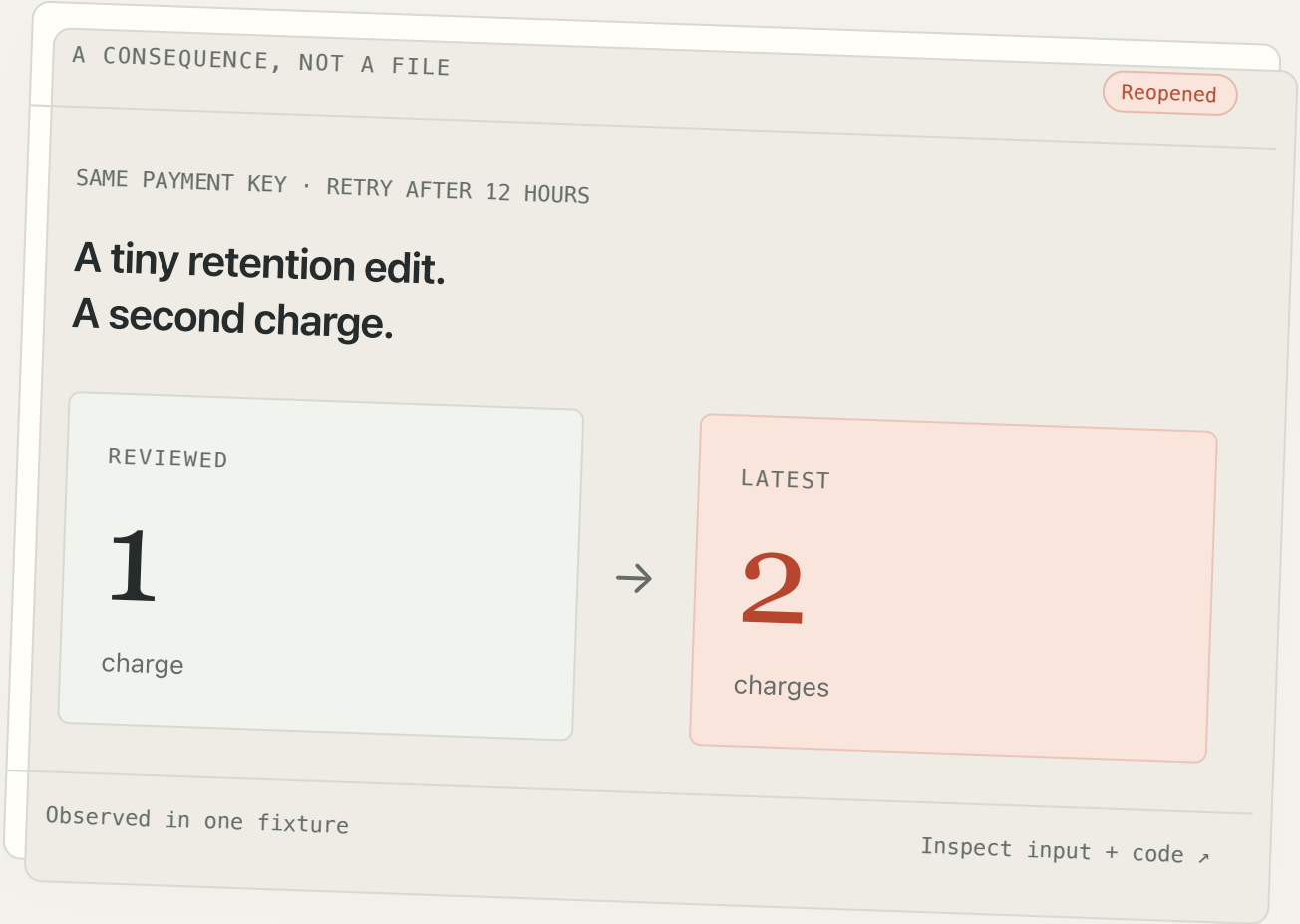
A local tool for the moment your agent finishes.
And the next time it finishes.

Try the interaction ↗

Start with the research →

LOCAL GIT + GITHUB EXAMPLES, THEN CODE

A researched product proposal. Not an implemented verifier. All examples in this deck are simulated.



The obvious ideas are *already here*.

Keep the strengths of modern review tools. Do not mistake a new arrangement of familiar features for an invention.

01 **Explain the change**
Linear Guides · Graphite Code Tours · CodeRabbit Change Stack

02 **Organize local agent work**
hunk-guide · Limn · Differential · Agent Diff Visualizer

03 **Reduce noise, map the flow**
SemanticDiff · Diffcore

04 **Preserve how it happened**
Entire Checkpoints

05 **Compare real behavior**
HyperTest · ReGrade · decades of differential verification

06 **Bind evidence, withdraw stale conclusions**
Critique · Proof · Semctx

The opening is a better *human interaction*.

Not another summary.
Not another agent verdict.

One condition.

Two outcomes. A way to inspect.

DUPLICATE-PAYMENT PROTECTION

Observed · fixture

INPUT

Key pay_123 · retry delay 12h

REVIEWED V1

1

charge recorded

→

CANDIDATE V2

2

charges recorded

Limit of this observation

Sequential requests. Concurrency not tested.

Input · outputs · environment · source

Pin expectation ◇

- 1

Understand the difference
An example is more specific than “improves retry handling.”
- 2

Know why to believe it
Keep the observation, its provenance, and its limits together.
- 3

Keep the implementation close
Exact hunks and relevant unchanged code stay one action away.
- 4

Remember what matters
Pin “one charge for this example.” A broader guarantee needs broader evidence.

Glance. Inspect. *Keep your place.*

Try the three steps below.
No commands run. No repository is read.

after / payments / resilient-retries

Inspected v1 → latest v1 · demo

Contact sheet

Examples

Source

Review memory

3 SAMPLE GROUPS OF 8 · NOT A COMPLETENESS CLAIM

Duplicate-payment protection

One captured example agrees

Observed

Timeout recovery

Error → retry with a safe key

Observed

Out-of-order webhooks

No paired result available

Not checked

Not pinned

SAME KEY · RETRY AFTER 12H

This example still produces one charge.

INSPECTED v1

1 charge

Captured fixture · sequential requests only

LATEST v1

→ 1 charge

Start by pinning the 12-hour example.

17 unclassified hunks ↗

Interactive HTML sequence: pin the example → simulate a retention edit → see evidence become stale → simulate a paired run → inspect the new result.

04 / 12 · AFTER · PRODUCT CONCEPT · ALL DEMO DATA SIMULATED

Find the edge where behavior *changes*.

Browse the conditions, not just the files.

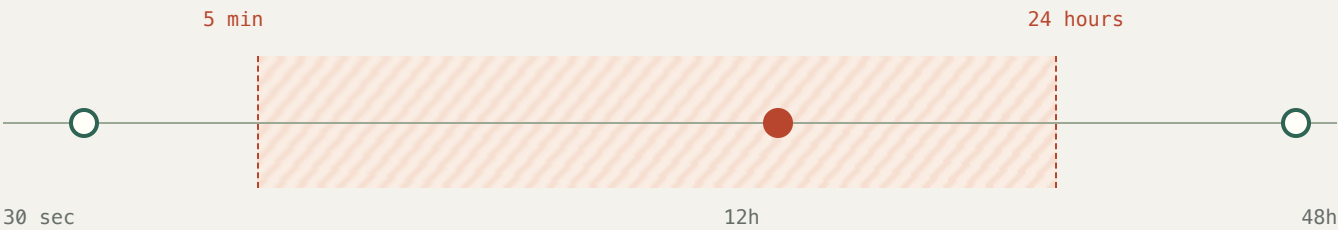
Fixed demo dataset · reviewed v1 versus candidate v2.
Same key. Same payment. Change only the retry delay.

30 seconds

12 hours

48 hours

2 hours · not run



● Captured points ▨ Inferred interval
Not to scale. Concurrency and clock skew not tested.

RETRY AFTER 2 HOURS

Not run

REVIEWED V1

?

charge count

→

CANDIDATE V2

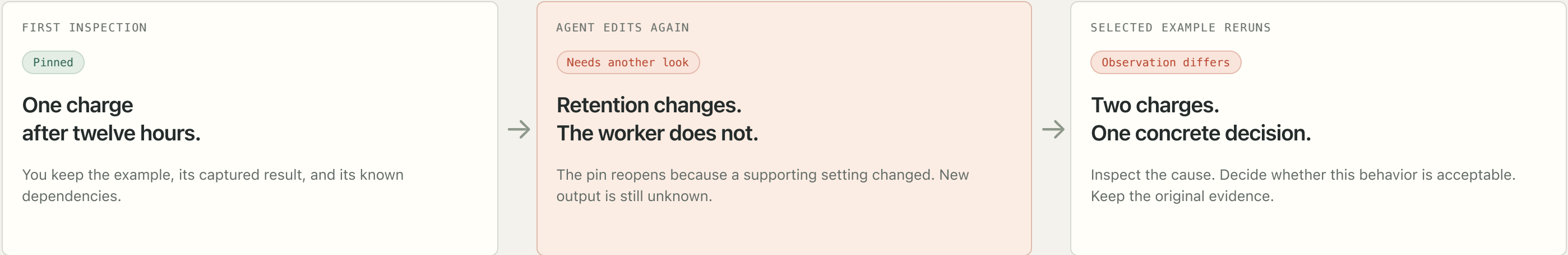
?

charge count

No captured result exists for this input. The shaded interval suggests a difference, but that is an inference, not an execution result.

Follow into the code →

Your attention should *accumulate*. Not reset.



Keep

History and decisions with unchanged known footprints.

Reopen

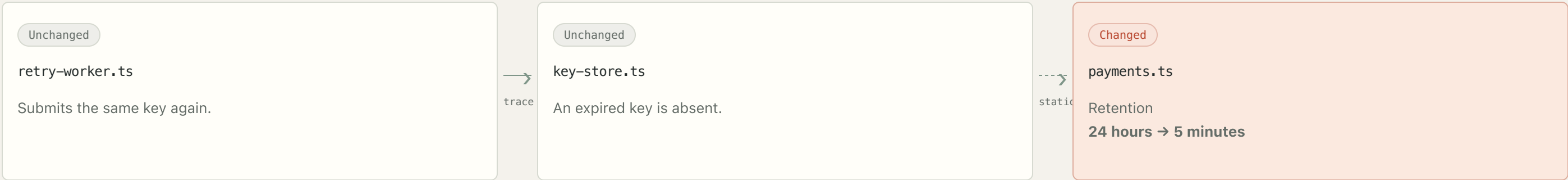
Expectations whose code, inputs, tools, or environment moved.

Say unknown

When the dependency inventory is incomplete.

The important line may be in *unchanged code*.

Start from an example. Expand a short path.
No whole-repository graph required.



THE VISIBLE EDIT

```
- KEY_RETENTION = hours(24)
+ KEY_RETENTION = minutes(5)
```

THE REVIEW QUESTION

Should the twelve-hour retry still reuse the first charge?

The example makes the consequence concrete.
The reviewer still decides the intended behavior.

Illustrative trace + static relationship. Inspect provenance before accepting a causal explanation.

No magic *green check.*

A readable story is not proof.
A passing example is not approval.

Observed

It ran for these inputs.
Keep outputs, environment, and snapshot. Do not generalize to all cases.

Structural

A bounded fact.
A value changed. An export disappeared. Runtime equivalence is not implied.

Inferred

A prediction to inspect.
A model or heuristic expects this result. No run is implied.

Author says

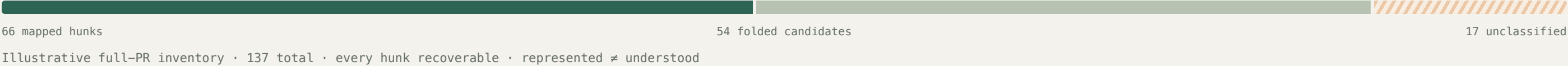
Intent, not verification.
Agent summaries and test claims retain their author-supplied status.

Not checked

The evidence is missing.
Unsupported languages, absent fixtures, and failed setup stay visible.

Stale

Its basis moved.
Keep the earlier observation. Withdraw its applicability to the new state.



Two entrances.

One review memory.

Local AFTER YOUR AGENT FINISHES

```
$ after
working tree · captured snapshot
```

- 1. Open without commits or agent logs.
- 2. Browse now. Add analysis as it arrives.
- 3. New edits notify; your snapshot stays still.

GitHub BEFORE YOU APPROVE

```
$ after pr 482
pinned base + head · no checkout switch
```

- 1. Read without touching your dirty worktree.
- 2. Inspect checks, examples, and exact changes.
- 3. Draft privately. Confirm before publishing.

Same example → same source anchors → explicit reconciliation

Proposed commands. No upload by default. Inference and execution require deliberate permission.

A useful browser.

Not a new engineering platform.

- 01

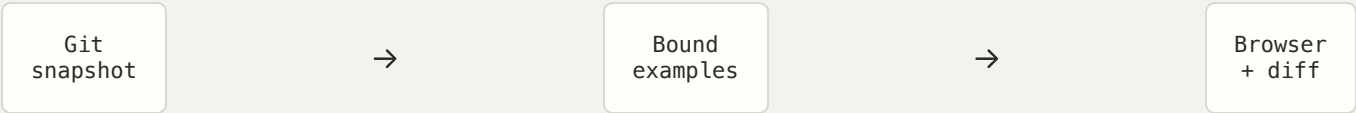
Git snapshot + ordinary diff
Local changes and read-only PR import. Useful before a model responds.
- 02

Contact sheet + existing examples
Start with TypeScript relationships and paired JSON/test artifacts.
- 03

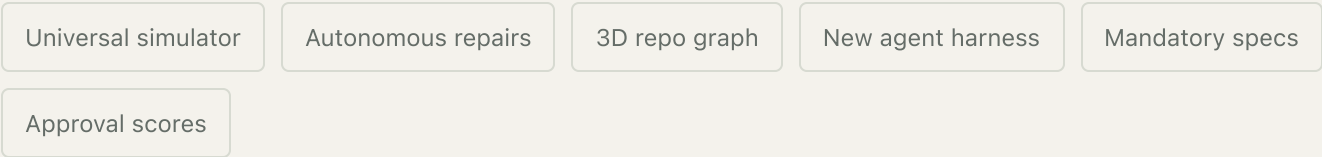
One-click pins + conservative reopening
Track exact evidence bindings. Show unknown when dependencies are incomplete.
- 04

Then add selected paired runs
Explicit permission. Isolated compute. No ambient credentials.

ONE SIMPLE SHAPE



DELIBERATELY NOT FIRST



Use hunk-guide to test the reading flow.
Do not bend Hunk's private APIs into a new UI.

The wedge is the *habit*.

Browse by example.
Pin an expectation.
Reopen it when its basis moves.

CLOSEST WORK	DOCUMENTED EMPHASIS	WHAT AFTER WOULD EMPHASIZE
Linear / CodeRabbit	Guided explanation of a PR	Concrete contrasts as the primary index
Limn	Local narrative + incremental review	Example bookmarks with evidence-aware reopening
HyperTest / ReGrade	Execution-backed behavioral comparisons	A human browsing surface across change types
Critique / Proof	Bound evidence, intent, verification state	Optional pins on ordinary diffs, without a required assurance workflow
AFTER hypothesis	Make those ideas feel like a fast diff browser, not an additional process.	

Easy to try. Worth reopening.

The defensible work is reliable bindings
and a remarkably direct interface.

Documentation-based positioning, not a claim that these capabilities are absent elsewhere.

A diff should show a consequence. Then *remember your judgment.*

Build the example-first interaction.
Prove that it saves attention before building the engine around it.

PRIMARY SOURCES · inspected October 2, 2026 · documentation review, not hands-on benchmarking

Linear Graphite CodeRabbit Limn Diffcore HyperTest ReGrade Critique Proof Semctx All 16 sources + limitations ↗

THE FIRST EXPERIMENT

Can people re-review faster without missing more?

Compare raw diffs, a strong guided tour, and AFTER on unfamiliar changes.

- Target ~30% less re-review time.
- No loss in defect detection.
- No silent freshness in adversarial tests.

Targets, not measured results.

If useful examples require months of setup, this is the wrong wedge. If the contact sheet works but pins do not, ship the contact sheet.